

① Searching (Linear Searching)

I/P: arr[] = {20, 5, 7, 25}

x = 5

O/P: 1

I/P: arr[] = {20, 5, 7, 25}

x = 15

O/P: -1

Algorithm

```
int Search(int arr[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == x)
            return i;
    return -1;
}
```

Time complexity: $O(n)$

Searching: Binary Search

Date:/...../.....
Page No.

Lec: 01 > Binary Search (iterative)

I/P: arr[] = {10, 20, 30, 40, 50, 60}
x = 20

O/P: 1

I/P: arr[] = {5, 10, 15, 25, 35}
x = 20

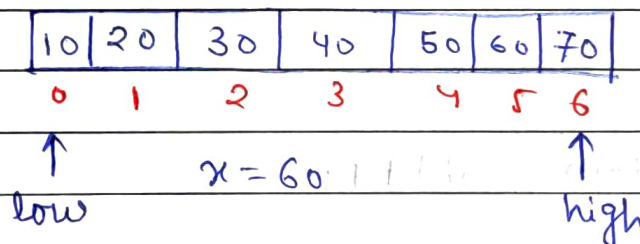
O/P: -1

I/P: arr[] = {5, 15, 25}
x = 25

O/P: 2

I/P: arr[] = {10, 10}
x = 10

O/P: 0 or 1



Steps:-

compute mid = (low + high) / 2

return mid ← case 1: (arr[mid] == x)

Repeat: high = mid - 1 ← case 2: (arr[mid] > x)

Repeat: low = mid + 1 ← case 3: (arr[mid] < x)

Dry Run:-

(i) mid = (0 + 6) / 2 = 3, arr[mid] = arr[3] = 40
∴ arr[mid] < x ⇒ low = mid + 1 = 3 + 1 = 4

(ii) mid = (4 + 6) / 2 = 5, arr[mid] = arr[5] = 60
∴ arr[mid] == x ⇒ return mid ⇒ return 5.

Function

```
int binarySearch(int arr[], int n, int x)
```

```
{
```

```
    int low = 0;
```

```
    int high = n - 1;
```

```
    while (low <= high)
```

```
    {
```

```
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == x)
```

```
        {
```

```
            return mid;
```

```
        }
```

```
        else if (arr[mid] > x)
```

```
        {
```

```
            high = mid - 1;
```

```
        }
```

```
        else
```

```
        {
```

```
            low = mid + 1;
```

```
        }
```

```
    }
```

```
    return -1;
```

```
}
```

Lec: 02 (Binary Search [Recursive])

Function

```
int binarySearch (int arr[], int low, int high, int x)
{
    if (low > high)
    {
        return -1;
    }

    int mid = (low + high) / 2;
    if (arr[mid] == x)
    {
        return mid;
    }
    else if (arr[mid] > x)
    {
        return binarySearch(arr, low, mid - 1, x);
    }
    else
    {
        return binarySearch(arr, mid + 1, high, x);
    }
}
```

① Example

10	20	30	40	50	60	70
----	----	----	----	----	----	----

0 1 2 3 4 5 6
 ↑ ↑ ↑ ↑ ↑ ↑ ↑
 low x = 20 high

1 → binarySearch(arr, 0, 6, 20)

 → mid = $(0 + 6) / 2 = 3$

 → arr[mid] > x

1 → binarySearch(arr, 0, 2, 20)

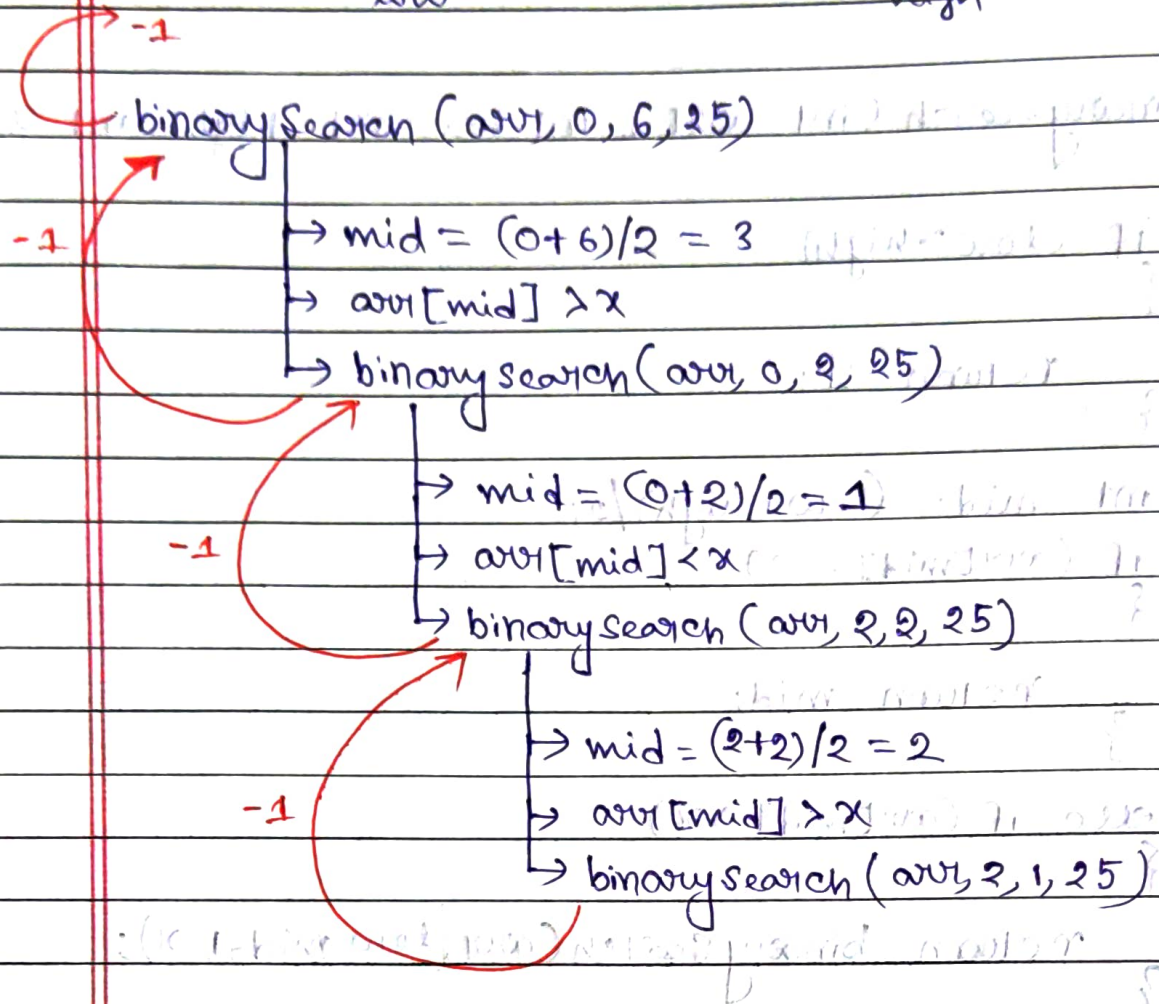
 → mid = $(0 + 2) / 2 = 1$

 → arr[mid] == x, return mid

arr =

10	20	30	40	50	60	70
0	1	2	3	4	5	6

low $x = 25$ high



Comparison b/w recursive & iterative binary search

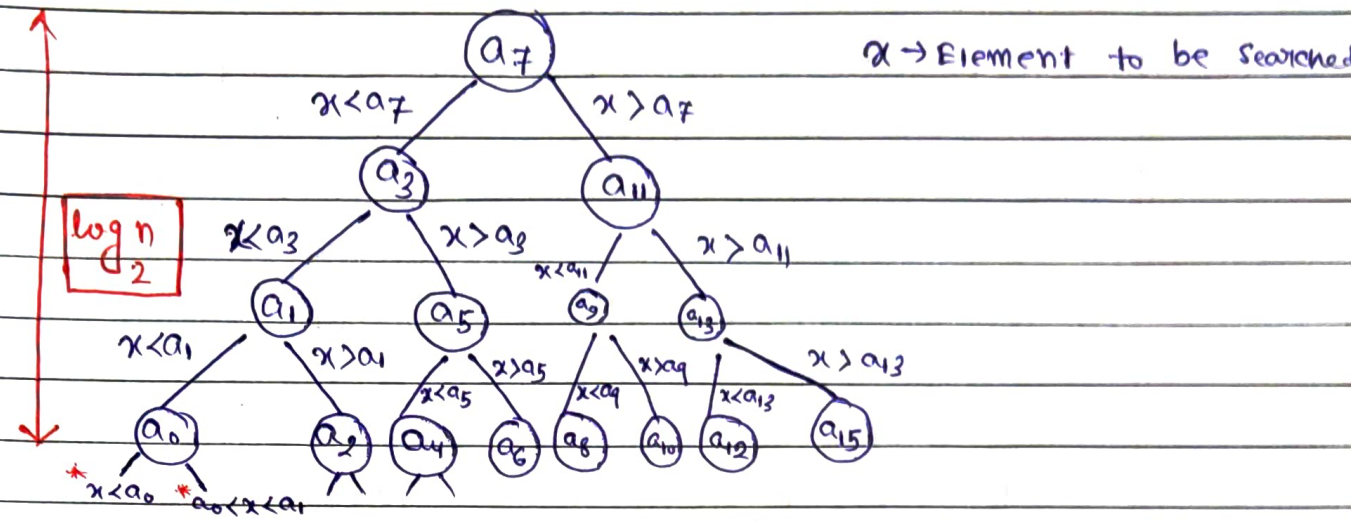
	Recursive	Iterative
Time comp.	$O(\log n)$	$O(\log n)$
Space comp.	$O(\log n)$	$O(1)$
Function call stack is to be maintained	Yes	No

Conclusion > Iterative is better than Recursive

Lec: 03) Analysis of Binary Search

arr

a_0	a_1	a_2		a_7	a_8		a_{14}	a_{15}
-------	-------	-------	-------	-------	-------	-------	----------	----------



$\rightarrow$ All possible execution path for Successful Searches.

**** $\rightarrow$ Unsuccessful Search.**

- ① Worst-case time complexity $\log n$
- ② Best-case time complexity $O(1)$